Aristotle University of Thessaloniki

MSc Program in Biomedical Engineering

Course BM020- Machine Learning in biomedical data analysis 2025

30/5/2025

# SKIN CANCER CLASSIFICATION

Using MLP-CNN

Professor: Panagiotis Petrantonakis
Students: Paraskevi Katsarela
Georgia Vetsika

# Περιεχόμενα

SKIN CANCER CLASSIFICATION – MULTILAYER PERCEPTRONS (MLP) ..... 2

SKIN CANCER CLASSIFICATION- CONVOLUTIONAL NEURAL NETWORK (CNN)..... 37

# SKIN CANCER CLASSIFICATION – MULTILAYER PERCEPTRONS (MLP)

1. **Data Loading and Setup:**

```
#Set up of the packages that will be needed
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical
from sklearn.decomposition import PCA
```

- NumPy is used for loading and manipulating the .npz files (compressed NumPy archive).
- Matplotlib is for plotting and visualizing images.
- to_categorical helps in one-hot encoding the labels (used in classification).

2. **Load and Explore the Data:**

```
# Load the training and testing data
train_data = np.load('DermaMNIST_Train.npz')
test_data = np.load('DermaMNIST_Test.npz')

#Accessing data keys for the input and the output
print("Train data keys:", train_data.files)
print("Test data keys:", test_data.files)

x_train = train_data['x_train']
y_train = train_data['y_train']
x_test = test_data['x_test']
y_test = test_data['y_test']
```

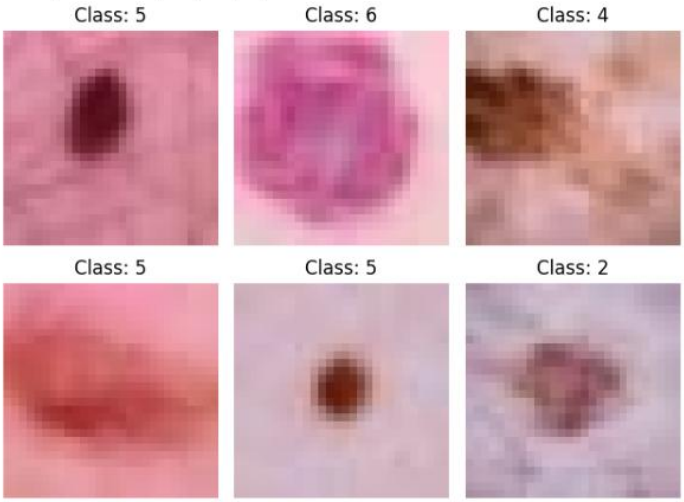
- .npz files contain pre-processed images and labels.
- You extract training and test data (x) and labels (y).

3. **Visual Inspection and Dataset Shape:**

```
#Displaying 6 sample images from training dataset with their corresponding class labels
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
print("Training labels shape:", y_train.shape)

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(x_train[i])
    plt.title(f"Class: {np.argmax(y_train[i])}")
    plt.axis('off')
plt.tight_layout()
plt.show()

unique, counts = np.unique(y_train, return_counts=True)
print(dict(zip(unique, counts)))
```

- This step prints the shape of the image data and value ranges to understand. Whether normalization is needed and what the input shape will be (e.g. 28x28 images).
- Visualizes 6 random training samples with their label (class) using imshow, as shown in the below picture:

```
Train data keys: ['x_train', 'y_train']
Test data keys: ['x_test', 'y_test']
Pixel value range: 0.0 to 1.0
Image shape: (28, 28, 3)
Training data shape: (7007, 28, 28, 3)
Testing data shape: (2005, 28, 28, 3)
Training labels shape: (7007, 7)
```




4. **Label Flattening & Reshaping for MLP:**

```
y_train_labels = np.argmax(y_train, axis=1)
x_train_flat = x_train.reshape((x_train.shape[0], -1))
x_test_flat = x_test.reshape((x_test.shape[0], -1))
```

- Converts labels from one-hot to class indices for computing class weights.
- MLPs require 1D input, so each 28x28 image is flattened to a 784-length vector.

5. **Class Imbalance Handling with Class Weights:**

```
#Compute Class weights

from sklearn.utils.class_weight import compute_class_weight
import numpy as np

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_labels),
    y=y_train_labels
)

# Create dictionary for Keras
class_weights_dict = dict(enumerate(class_weights))

print("Class Weights:", class_weights_dict)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, BatchNormalization

# Number of classes
num_classes = len(np.unique(y_train_labels))
```

- Medical datasets often have imbalanced classes. compute_class_weight balances the importance of each class during training.
- class_weights_dict will later be used in model training.

6. **One-hot Encoding:**

```
y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test
```

- Converts flat labels back to one-hot format, which is the required output format when using categorical_crossentropy as a loss function.

7. **MLP Model Design (example of 3 layers):**

```
# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

- Architecture: 3 hidden dense layers, with relu activation and dropout (to reduce overfitting).
- Output layer: softmax, suitable for multi-class classification.
- This is a fully connected network, an MLP with progressively decreasing number of neurons (512 → 256 → 128 → output).

## 8. Compilation & Training of the model:

```
#Compile model
mlp_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

mlp_model.fit(x_train_flat, y_train_oh, epochs=20, batch_size=64, validation_split=0.1)

#Test accuracy
test_loss, test_acc = mlp_model.evaluate(x_test_flat, y_test_oh, verbose=0)
print(f"MLP Test accuracy: {test_acc:.4f}")
```

- Optimizer: Adam is used for adaptive learning rate.
- Loss: categorical_crossentropy because it's multi-class classification.
- Validation Split: 10% of training data used for validation during training.
- Epochs & Batch Size: Reasonable defaults (10 epochs, batch size 64).
- Test accuracy evaluation: Prints the overall test accuracy of the model

## CASES

The goal of this section is to design and evaluate a Multilayer Perceptron (MLP) for classifying skin cancer images from the DermaMNIST dataset using supervised learning. This involved experimenting with the network architecture and training parameters to optimize classification accuracy while minimizing overfitting.

The data was loaded from .npz files containing:

- Training images: x_train (shape: 7007, 28, 28)
- One-hot encoded labels: y_train (shape: 7007, 2)
- Test images and labels in a similar format

Data preprocessing included:

- Flattening the 2D image arrays into 1D vectors (28x28 = 784 features)
- Ensuring labels were correctly one-hot encoded for categorical cross-entropy loss

**Trial 1: MLP 3-LAYERS**

a) The first MLP model was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

**Parameters:** Optimizer →Adam, loss →Categorical cross-entropy, epochs →10, batch size →64, validation split → 10%

**Dense Layers:** Chosen to capture complex patterns in flattened image data. The architecture gradually reduces the dimensionality (512 → 256 → 128), which helps in feature compression while retaining relevant information.

**Dropout Layers:** Added after each dense layer to reduce overfitting by randomly deactivating 30% of neurons during training. Dropout helps generalize better to unseen data (test set).

**Activation Functions:** ReLU was used in hidden layers due to its simplicity and effectiveness. Softmax was used in the output layer for multi-class classification (in this case, binary classification with 2 output neurons).

**Evaluation Results:**

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 28ms/step - accuracy: 0.5955 - loss: 1.4740 - val_accuracy: 0.6633 - val_loss: 1.0924
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 37ms/step - accuracy: 0.6719 - loss: 1.0753 - val_accuracy: 0.6633 - val_loss: 1.0224
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 26ms/step - accuracy: 0.6711 - loss: 1.0565 - val_accuracy: 0.6633 - val_loss: 1.0008
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - accuracy: 0.6736 - loss: 1.0099 - val_accuracy: 0.6633 - val_loss: 0.9861
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.6739 - loss: 0.9945 - val_accuracy: 0.6633 - val_loss: 0.9944
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 38ms/step - accuracy: 0.6775 - loss: 1.0155 - val_accuracy: 0.6633 - val_loss: 1.0008
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 31ms/step - accuracy: 0.6768 - loss: 1.0045 - val_accuracy: 0.6633 - val_loss: 0.9970
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.6658 - loss: 1.0246 - val_accuracy: 0.6633 - val_loss: 0.9977
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.6813 - loss: 0.9927 - val_accuracy: 0.6633 - val_loss: 1.0029
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.6761 - loss: 0.9937 - val_accuracy: 0.6633 - val_loss: 0.9802
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0924 to 0.9802 over 10 epochs |
| Final Parameters | ~1.37 million |

Table 1. First MLP model-3layers results.

The results in Table 1 indicate maybe an overfitting occurrence or a plateau (due to the static val_accuracy). Validation accuracy tracks closely with training accuracy. The confusion between class weights and data balance was also handled using compute_class_weight from sklearn, to ensure that minority classes were properly weighted during training. Val_loss seems to decrease but it is unstable.

b) The second MLP model was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.4),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

In the MLP layers, the dropout in the first layer 512 is slightly increased from 0.3 → 0.4. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - accuracy: 0.5967 - loss: 1.4469 - val_accuracy: 0.6633 - val_loss: 1.0626
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - accuracy: 0.6741 - loss: 1.1175 - val_accuracy: 0.6633 - val_loss: 1.0559
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - accuracy: 0.6576 - loss: 1.0670 - val_accuracy: 0.6633 - val_loss: 1.0469
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.6657 - loss: 1.0685 - val_accuracy: 0.6633 - val_loss: 1.0515
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.6845 - loss: 1.0525 - val_accuracy: 0.6633 - val_loss: 1.0237
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.6647 - loss: 1.0831 - val_accuracy: 0.6633 - val_loss: 1.0436
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.6693 - loss: 1.0642 - val_accuracy: 0.6633 - val_loss: 1.0386
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.6699 - loss: 1.0356 - val_accuracy: 0.6633 - val_loss: 1.0192
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 36ms/step - accuracy: 0.6785 - loss: 1.0204 - val_accuracy: 0.6633 - val_loss: 1.0351
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.6707 - loss: 1.0328 - val_accuracy: 0.6633 - val_loss: 1.0473
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0626 to 1.0192 over 10 epochs in an unstable way |
| Final Parameters | ~1.37 million |

Table 2. Second MLP model-3layers results.

The results in Table 2 indicate maybe an overfitting occurrence or a plateau (due to the static val_accuracy). Validation accuracy tracks closely with training accuracy. Val_accuracy and test accuracy are the same as before, but val_loss is a little bit worse than the previous trial.

c) The third MLP model was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.4),
    Dense(256, activation='relu'),
    Dropout(0.4),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

In the MLP layers, the dropout in the first layer 512 is 0.4. Now in layer 256, the dropout is slightly increased from 0.3 → 0.4. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 28ms/step - accuracy: 0.5922 - loss: 1.4980 - val_accuracy: 0.6633 - val_loss: 1.0664
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - accuracy: 0.6673 - loss: 1.0972 - val_accuracy: 0.6633 - val_loss: 1.0819
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - accuracy: 0.6792 - loss: 1.0836 - val_accuracy: 0.6633 - val_loss: 1.0627
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - accuracy: 0.6707 - loss: 1.0423 - val_accuracy: 0.6633 - val_loss: 1.0303
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - accuracy: 0.6761 - loss: 1.0356 - val_accuracy: 0.6633 - val_loss: 1.0366
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.6713 - loss: 1.0346 - val_accuracy: 0.6633 - val_loss: 1.0229
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - accuracy: 0.6735 - loss: 1.0461 - val_accuracy: 0.6633 - val_loss: 1.0349
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.6730 - loss: 1.0404 - val_accuracy: 0.6633 - val_loss: 1.0060
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.6590 - loss: 1.0446 - val_accuracy: 0.6633 - val_loss: 1.0224
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 6s 38ms/step - accuracy: 0.6715 - loss: 1.0350 - val_accuracy: 0.6633 - val_loss: 1.0148
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0664 to 1.0060 over 10 epochs in an unstable way |
| Final Parameters | ~1.37 million |

Table 3. Third MLP model-3layers results.

The results in Table 3 indicate maybe an overfitting occurrence or a plateau (due to the static val_accuracy). Validation accuracy tracks closely with training accuracy. Val_accuracy and test accuracy are the same as before, but accuracy seems to have a slight drop.

d) The fourth MLP model was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.5),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

In the MLP layers, the dropout in the first layer 512 is slightly increased from 0.4 → 0.5. In layer 256, the dropout is slightly increased from 0.4 → 0.3. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 34ms/step - accuracy: 0.5747 - loss: 1.5610 - val_accuracy: 0.6633 - val_loss: 1.0712
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - accuracy: 0.6750 - loss: 1.1089 - val_accuracy: 0.6633 - val_loss: 1.0651
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.6673 - loss: 1.0846 - val_accuracy: 0.6633 - val_loss: 1.0461
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.6664 - loss: 1.0699 - val_accuracy: 0.6633 - val_loss: 1.0432
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 36ms/step - accuracy: 0.6683 - loss: 1.0737 - val_accuracy: 0.6633 - val_loss: 1.0554
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.6684 - loss: 1.0402 - val_accuracy: 0.6633 - val_loss: 1.0268
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 25ms/step - accuracy: 0.6669 - loss: 1.0429 - val_accuracy: 0.6633 - val_loss: 1.0565
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.6647 - loss: 1.0470 - val_accuracy: 0.6633 - val_loss: 1.0158
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.6666 - loss: 1.0439 - val_accuracy: 0.6633 - val_loss: 1.0578
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 36ms/step - accuracy: 0.6782 - loss: 1.0248 - val_accuracy: 0.6633 - val_loss: 1.0128
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0712 to 1.0128 over 10 epochs in an unstable way |
| Final Parameters | ~1.37 million |

Table 4. Fourth MLP model-3layers results.

The results in Table 4 indicate maybe an overfitting occurrence or a plateau (due to the static val_accuracy).Validation accuracy tracks closely with training accuracy. Furthermore, the training is stable, because loss steadily decreased without divergence or instability. Accuracy again has an unstable pattern increase, containing also dropping of the value. Results are almost identical previous results.

e) The fifth MLP model was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

In the MLP layers, the dropout in the first layer 512 is slightly decreased to 0.3. In layer 256, the dropout is at 0.3, but layer 128 has now 0.2 dropout. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.5934 - loss: 1.3810 - val_accuracy: 0.6633 - val_loss: 1.0774
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - accuracy: 0.6693 - loss: 1.0698 - val_accuracy: 0.6633 - val_loss: 1.0158
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.6686 - loss: 1.0475 - val_accuracy: 0.6633 - val_loss: 1.0132
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 37ms/step - accuracy: 0.6632 - loss: 1.0201 - val_accuracy: 0.6633 - val_loss: 1.0331
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - accuracy: 0.6707 - loss: 1.0299 - val_accuracy: 0.6633 - val_loss: 1.0012
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - accuracy: 0.6702 - loss: 0.9887 - val_accuracy: 0.6633 - val_loss: 0.9858
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 35ms/step - accuracy: 0.6753 - loss: 0.9915 - val_accuracy: 0.6633 - val_loss: 0.9758
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 30ms/step - accuracy: 0.6693 - loss: 0.9814 - val_accuracy: 0.6633 - val_loss: 1.0152
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.6731 - loss: 1.0214 - val_accuracy: 0.6633 - val_loss: 1.0653
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.6742 - loss: 1.0235 - val_accuracy: 0.6633 - val_loss: 0.9932
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0774 to 0.9932 over 10 epochs in an unstable way |
| Final Parameters | ~1.37 million |

Table 5. Fifth MLP model-3layers results.

The results in Table 5 indicate maybe an overfitting occurrence or a plateau (due to the static val_accuracy). Validation accuracy tracks closely with training accuracy. Results follow the same pattern as before.

f) The sixth MLP model was built using the following structure:

```python
# Build MLP model
mlp_model = Sequential([
    Dense(256, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

In this model, architecture gradually reduces the dimensionality from 512→256→128 to 256→128→64, resulting in a slight drop in accuracy, indicating that higher capacity was beneficial. Dropout also decreased to 0.3 & 0.2. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - accuracy: 0.6040 - loss: 1.3464 - val_accuracy: 0.6633 - val_loss: 1.0371
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 0.6756 - loss: 1.0426 - val_accuracy: 0.6633 - val_loss: 1.0150
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.6772 - loss: 1.0399 - val_accuracy: 0.6633 - val_loss: 1.0030
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.6724 - loss: 1.0026 - val_accuracy: 0.6633 - val_loss: 1.0053
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 0.6665 - loss: 1.0109 - val_accuracy: 0.6633 - val_loss: 0.9716
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 21ms/step - accuracy: 0.6615 - loss: 1.0201 - val_accuracy: 0.6633 - val_loss: 0.9756
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.6770 - loss: 0.9910 - val_accuracy: 0.6633 - val_loss: 0.9925
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 0.6759 - loss: 0.9934 - val_accuracy: 0.6633 - val_loss: 1.0031
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.6686 - loss: 1.0220 - val_accuracy: 0.6633 - val_loss: 0.9792
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - accuracy: 0.6615 - loss: 0.9902 - val_accuracy: 0.6633 - val_loss: 0.9702
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0774 to 0.9932 over 10 epochs in an unstable way |
| Final Parameters | 643,975 |

Table 6. Sixth MLP model-3layers results.

The results in Table 6 indicate maybe an overfitting occurrence or a plateau (due to the static val_accuracy). Validation accuracy tracks closely with training accuracy. On the other hand, the final parameters used in this model are less than in the previous model (~1.37 million → ~644 hundreds). This reduction happens because each dense layer's parameters are calculated as: (number of inputs × number of neurons) + number of biases. So, this model calculates less neurons, the connections are also fewer, and it results in less parameters, almost half of the previous models.

g) In the next 2 trials, a new order is added in the code: BatchNormalization. This adds stability to training and often improves validation performance a bit (no extra training time). The seventh MLP model was built using the following structure:

```python
# Build MLP model
from tensorflow.keras.layers import BatchNormalization

mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),

    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality and dropout are kept the same as the fifth model. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 44ms/step - accuracy: 0.4167 - loss: 2.0110 - val_accuracy: 0.4693 - val_loss: 1.8759
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 41ms/step - accuracy: 0.6163 - loss: 1.1936 - val_accuracy: 0.6662 - val_loss: 1.0886
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 39ms/step - accuracy: 0.6574 - loss: 0.9919 - val_accuracy: 0.6262 - val_loss: 0.9706
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 46ms/step - accuracy: 0.6910 - loss: 0.9047 - val_accuracy: 0.6220 - val_loss: 1.1229
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 43ms/step - accuracy: 0.6826 - loss: 0.8717 - val_accuracy: 0.6690 - val_loss: 1.1308
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 34ms/step - accuracy: 0.6898 - loss: 0.8745 - val_accuracy: 0.6619 - val_loss: 1.1671
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 39ms/step - accuracy: 0.6939 - loss: 0.8663 - val_accuracy: 0.5278 - val_loss: 1.0796
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 52ms/step - accuracy: 0.6901 - loss: 0.8336 - val_accuracy: 0.6576 - val_loss: 1.0523
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 47ms/step - accuracy: 0.6975 - loss: 0.8037 - val_accuracy: 0.6491 - val_loss: 1.4519
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 53ms/step - accuracy: 0.7049 - loss: 0.8057 - val_accuracy: 0.6491 - val_loss: 1.4951
MLP Test accuracy: 0.6554
```

| Training Summary | |
|---|---|
| Testing Accuracy | 65.54% |
| Validation Accuracy | 64.91% |
| Validation Loss | Decreased from 1.8759 to 1.4951 over 10 epochs in an unstable way |
| Final Parameters | 1.3 million |

Table 7. Seventh MLP model-3layers results.

The results in Table 7 indicate no overfitting. Validation accuracy is no longer stable, it progressively increases in an unstable manner and afterwards there is a small drop.

The eighth MLP model was built using the following structure, with BatchNormalization:

```python
# Build MLP model
from tensorflow.keras.layers import BatchNormalization

mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(x_train_flat.shape[1],)),
    BatchNormalization(),
    Dropout(0.2),

    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.1),

    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality is kept the same as the seventh model. But dropouts are slightly decreased to 0.2, 0.2, 0.1 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 31ms/step - accuracy: 0.4354 - loss: 1.8558 - val_accuracy: 0.6334 - val_loss: 1.8853
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - accuracy: 0.6517 - loss: 1.0982 - val_accuracy: 0.6748 - val_loss: 1.2234
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.6833 - loss: 0.9199 - val_accuracy: 0.6790 - val_loss: 0.9212
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.6902 - loss: 0.8716 - val_accuracy: 0.6690 - val_loss: 1.1804
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 6s 41ms/step - accuracy: 0.6969 - loss: 0.8244 - val_accuracy: 0.6676 - val_loss: 1.2899
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - accuracy: 0.7007 - loss: 0.8087 - val_accuracy: 0.6576 - val_loss: 0.9417
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 29ms/step - accuracy: 0.7157 - loss: 0.7832 - val_accuracy: 0.5506 - val_loss: 1.1309
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - accuracy: 0.7028 - loss: 0.7884 - val_accuracy: 0.6462 - val_loss: 1.5526
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.7106 - loss: 0.7688 - val_accuracy: 0.6662 - val_loss: 1.6682
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.7166 - loss: 0.7653 - val_accuracy: 0.5849 - val_loss: 1.5687
MLP Test accuracy: 0.5875
```

| Training Summary | |
|---|---|
| Testing Accuracy | 58.75% |
| Validation Accuracy | 58.49% (last epoch) |
| Validation Loss | Decreased from 1.8853 to 1.5687 over 10 epochs in an unstable way |
| Final Parameters | 1.3 million |

Table 8. Eighth MLP model-3layers results.

The results in Table 8 indicate no overfitting. Validation accuracy tracks closely with training accuracy. The results in Table 7 indicate no overfitting. Validation accuracy is no longer stable, it progressively increases in an unstable manner and afterwards there is a small drop.

**Trial 2: MLP 4-LAYERS**

The models that are following have 4 layers with multiple dimensionality and dropout combinations.

a) The first model of the MLP 4Layers cases was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.4),
    Dense(1024, activation='relu'),
    Dropout(0.3),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality follows this architecture 2048→1024→512→256, and dropout 0.4→0.3→0.3→0.3 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 16s 140ms/step - accuracy: 0.5852 - loss: 1.6227 - val_accuracy: 0.6633 - val_loss: 1.1710
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 14s 139ms/step - accuracy: 0.6639 - loss: 1.1177 - val_accuracy: 0.6633 - val_loss: 1.0236
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 135ms/step - accuracy: 0.6703 - loss: 1.0691 - val_accuracy: 0.6633 - val_loss: 1.0391
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 14s 136ms/step - accuracy: 0.6672 - loss: 1.0338 - val_accuracy: 0.6633 - val_loss: 1.0095
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 135ms/step - accuracy: 0.6709 - loss: 1.0309 - val_accuracy: 0.6633 - val_loss: 0.9891
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 140ms/step - accuracy: 0.6651 - loss: 1.0442 - val_accuracy: 0.6633 - val_loss: 1.0039
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 145ms/step - accuracy: 0.6779 - loss: 1.0232 - val_accuracy: 0.6633 - val_loss: 1.0087
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 137ms/step - accuracy: 0.6619 - loss: 1.0625 - val_accuracy: 0.6633 - val_loss: 1.0099
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 136ms/step - accuracy: 0.6690 - loss: 1.0524 - val_accuracy: 0.6633 - val_loss: 1.0161
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 136ms/step - accuracy: 0.6583 - loss: 1.0623 - val_accuracy: 0.6633 - val_loss: 1.0367
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.710 to 1.0367 over 10 epochs in an unstable way |
| Final Parameters | ~7.6 million |

Table 9. First MLP model-4layers results.

The results in Table 9 are once again unstable. Validation accuracy tracks closely with training accuracy, very close to the training accuracy from 3layers models. On the other hand, the final parameters used in this model are more than in the previous models (~1.37 million → ~7.6 million). This increase happens because each Dense layer's parameters are calculated as: (number of inputs × number of neurons) + number of biases. So, this model calculates more neurons, the

connections are also more, and it results in more parameters, almost 4 times more than in the previous models.

b) The second model of the MLP 4Layers cases was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.3),
    Dense(1024, activation='relu'),
    Dropout(0.2),
    Dense(512, activation='relu'),
    Dropout(0.2),
    Dense(256, activation='relu'),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality follows the same architecture, and dropout 0.3→0.2→0.2→0.2 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 17s 143ms/step - accuracy: 0.6132 - loss: 1.5776 - val_accuracy: 0.6633 - val_loss: 1.0407
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 14s 139ms/step - accuracy: 0.6643 - loss: 1.0785 - val_accuracy: 0.6633 - val_loss: 1.0298
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 14s 142ms/step - accuracy: 0.6746 - loss: 1.0202 - val_accuracy: 0.6633 - val_loss: 1.0898
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 137ms/step - accuracy: 0.6676 - loss: 1.0309 - val_accuracy: 0.6633 - val_loss: 0.9698
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 22s 149ms/step - accuracy: 0.6757 - loss: 1.0163 - val_accuracy: 0.6633 - val_loss: 0.9727
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 142ms/step - accuracy: 0.6695 - loss: 0.9944 - val_accuracy: 0.6633 - val_loss: 1.0201
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 136ms/step - accuracy: 0.6590 - loss: 1.0396 - val_accuracy: 0.6633 - val_loss: 0.9902
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 140ms/step - accuracy: 0.6658 - loss: 1.0062 - val_accuracy: 0.6633 - val_loss: 0.9966
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 136ms/step - accuracy: 0.6661 - loss: 1.0007 - val_accuracy: 0.6633 - val_loss: 0.9772
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 134ms/step - accuracy: 0.6890 - loss: 0.9691 - val_accuracy: 0.6633 - val_loss: 1.0195
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0407 to 1.0195 over 10 epochs in an unstable way |
| Final Parameters | ~7.6 million |

Table 10. Second MLP model-4layers results.

The results in Table 10 are once again unstable. Validation accuracy tracks closely with training accuracy, which is very close to the training accuracy from 3layers models. Accuracy results increase and decrease through the epochs.

The third model of the MLP 4Layers cases was built using the following structure:

```python
# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.2),
    Dense(1024, activation='relu'),
    Dropout(0.2),
    Dense(512, activation='relu'),
    Dropout(0.2),
    Dense(256, activation='relu'),
    Dropout(0.1),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality follows the same architecture, and dropout is decreased to 0.2→0.2→0.2→0.1 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 17s 147ms/step - accuracy: 0.5942 - loss: 1.5649 - val_accuracy: 0.6633 - val_loss: 1.0744
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 14s 136ms/step - accuracy: 0.6600 - loss: 1.0724 - val_accuracy: 0.6633 - val_loss: 1.0155
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 132ms/step - accuracy: 0.6749 - loss: 1.0121 - val_accuracy: 0.6633 - val_loss: 1.0630
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 132ms/step - accuracy: 0.6660 - loss: 1.0221 - val_accuracy: 0.6633 - val_loss: 1.0344
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 13s 131ms/step - accuracy: 0.6758 - loss: 0.9746 - val_accuracy: 0.6633 - val_loss: 0.9781
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 139ms/step - accuracy: 0.6723 - loss: 0.9657 - val_accuracy: 0.6633 - val_loss: 0.9904
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 136ms/step - accuracy: 0.6722 - loss: 0.9728 - val_accuracy: 0.6633 - val_loss: 1.0711
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 130ms/step - accuracy: 0.6644 - loss: 1.0098 - val_accuracy: 0.6633 - val_loss: 0.9582
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 13s 130ms/step - accuracy: 0.6611 - loss: 0.9908 - val_accuracy: 0.6633 - val_loss: 0.9482
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 128ms/step - accuracy: 0.6708 - loss: 0.9843 - val_accuracy: 0.6633 - val_loss: 1.0143
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.0744 to 1.0143 over 10 epochs in an unstable way |
| Final Parameters | ~7.6 million |

Table 11. Third MLP model-4layers results.

The results have a similar pattern to before. Nothing is noticeably improving in the accuracy or val_loss.

c) The fourth model of the MLP 4Layers cases was built using the following structure:

```python
# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.5),
    Dense(1024, activation='relu'),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.4),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality follows the same architecture, and dropout is now increased to 0.5→0.4→0.4→0.3 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 16s 142ms/step - accuracy: 0.5747 - loss: 1.7329 - val_accuracy: 0.6633 - val_loss: 1.3795
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 139ms/step - accuracy: 0.6756 - loss: 1.1251 - val_accuracy: 0.6633 - val_loss: 1.0311
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 149ms/step - accuracy: 0.6637 - loss: 1.0928 - val_accuracy: 0.6633 - val_loss: 1.0336
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 14s 146ms/step - accuracy: 0.6672 - loss: 1.0454 - val_accuracy: 0.6633 - val_loss: 1.0789
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 19s 133ms/step - accuracy: 0.6732 - loss: 1.0538 - val_accuracy: 0.6633 - val_loss: 1.1116
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 139ms/step - accuracy: 0.6690 - loss: 1.0717 - val_accuracy: 0.6633 - val_loss: 1.0113
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 147ms/step - accuracy: 0.6673 - loss: 1.0633 - val_accuracy: 0.6633 - val_loss: 1.0758
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 138ms/step - accuracy: 0.6767 - loss: 1.0261 - val_accuracy: 0.6633 - val_loss: 1.0494
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 136ms/step - accuracy: 0.6720 - loss: 1.0425 - val_accuracy: 0.6633 - val_loss: 1.0318
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 135ms/step - accuracy: 0.6716 - loss: 1.0293 - val_accuracy: 0.6633 - val_loss: 1.0423
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.3795 to 1.0423 over 10 epochs in an unstable way |
| Final Parameters | ~7.6 million |

Table 12. Fourth MLP model-4layers results.

The results in Table 12 have a similar pattern to before. Nothing is noticeably improving in accuracy or val_loss. The model was less accurate in the beginning (0.57 accuracy) making more mistakes on validation data. After training for 10 epochs, it increased at 0.67 in an unstable way. Those results indicate an unstable generalization, although it improves over 10 epochs.

d) The fifth model of the MLP 4Layers cases was built using the following structure:

```
# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.4),
    Dense(1024, activation='relu'),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.1),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality follows the same architecture, and dropout is now changed to 0.5→0.4→0.3→0.1 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 17s 138ms/step - accuracy: 0.6024 - loss: 1.6246 - val_accuracy: 0.6633 - val_loss: 1.1225
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 131ms/step - accuracy: 0.6783 - loss: 1.0585 - val_accuracy: 0.6633 - val_loss: 1.0344
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 22s 144ms/step - accuracy: 0.6768 - loss: 1.0275 - val_accuracy: 0.6633 - val_loss: 1.0196
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 13s 130ms/step - accuracy: 0.6718 - loss: 1.0291 - val_accuracy: 0.6633 - val_loss: 0.9900
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 13s 131ms/step - accuracy: 0.6706 - loss: 1.0335 - val_accuracy: 0.6633 - val_loss: 1.0358
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 133ms/step - accuracy: 0.6573 - loss: 1.0945 - val_accuracy: 0.6633 - val_loss: 1.0176
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 133ms/step - accuracy: 0.6684 - loss: 1.0434 - val_accuracy: 0.6633 - val_loss: 1.0219
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 132ms/step - accuracy: 0.6609 - loss: 1.0673 - val_accuracy: 0.6633 - val_loss: 1.0198
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 21s 135ms/step - accuracy: 0.6689 - loss: 1.0330 - val_accuracy: 0.6633 - val_loss: 1.0514
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 20s 133ms/step - accuracy: 0.6669 - loss: 1.0572 - val_accuracy: 0.6633 - val_loss: 1.0289
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.1225 to 1.0289 over 10 epochs in an unstable way |
| Final Parameters | ~7.6 million |

Table 13. Fifth MLP model-4layers results.

The results in Table 13 follow a similar pattern as before. Val_accuracy is stable and training accuracy increases until it hits a plateau where it starts to decrease. Val_loss gradually decreases in an unstable way.

**Trial 3: MLP 7-LAYERS**

The model that follows has 7 layers dimensionality.

```python
# Build MLP model
mlp_model = Sequential([
    Dense(4096, activation='relu', input_shape=(x_train_flat.shape[1],)),
    Dropout(0.4),
    Dense(2048, activation='relu'),
    Dropout(0.3),
    Dense(1024, activation='relu'),
    Dropout(0.3),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The dimensionality follows this architecture:

4096→2048→1024→512→256→128→64, and dropout:

0.4→0.3→0.3→0.3→0.3→0.3→0.3 respectively. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 52s 481ms/step - accuracy: 0.5542 - loss: 1.7140 - val_accuracy: 0.6633 - val_loss: 1.1850
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 50s 501ms/step - accuracy: 0.6730 - loss: 1.1274 - val_accuracy: 0.6633 - val_loss: 1.0448
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 80s 477ms/step - accuracy: 0.6729 - loss: 1.0858 - val_accuracy: 0.6633 - val_loss: 1.0356
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 82s 479ms/step - accuracy: 0.6735 - loss: 1.0592 - val_accuracy: 0.6633 - val_loss: 1.0168
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 83s 483ms/step - accuracy: 0.6792 - loss: 1.0412 - val_accuracy: 0.6633 - val_loss: 1.0271
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 48s 481ms/step - accuracy: 0.6715 - loss: 1.0833 - val_accuracy: 0.6633 - val_loss: 1.1317
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 81s 473ms/step - accuracy: 0.6759 - loss: 1.0576 - val_accuracy: 0.6633 - val_loss: 1.0127
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 48s 484ms/step - accuracy: 0.6664 - loss: 1.0579 - val_accuracy: 0.6633 - val_loss: 1.0221
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 81s 477ms/step - accuracy: 0.6511 - loss: 1.0836 - val_accuracy: 0.6633 - val_loss: 1.0268
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 47s 478ms/step - accuracy: 0.6738 - loss: 1.0512 - val_accuracy: 0.6633 - val_loss: 1.0345
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Training Accuracy | 66.88% |
| Validation Accuracy | 66.33% |
| Validation Loss | Decreased from 1.1850 to 1.0345 over 10 epochs |
| Final Parameters | ~20.8 million |

Table 14. MLP model-7layers results.

The 7-layer MLP model recorded high training accuracy which remained for most of the epochs, indicating better learning and generalization but it also showed an instability with lowering the accuracy value at times. The validation loss decreased steadily from 1.1850 to 1.0168 over the first 4 epochs and then started to increase. Although the model contained a very large number of parameters (~20.8 million), the presence of dropout layers in every stage likely provided regularization and a better performance stability.

## CONCLUSION

Even though multiple combinations of models were conducted, with variation of dimensionality and dropout, we conclude the following:

- Validation accuracy is stable at 66.33%, which indicates that the MLP models have either reached a plateau of learning from the dataset or it is a sign of overfitting.
- Validation accuracy consistently tracked closely with training accuracy. The fact that, in general, validation accuracy slightly increases during the epochs shows that the model is learning something and trying to generalize but as epochs run this becomes difficult.
- Validation loss decreased over 10 epochs in all models with some fluctuations.
- Slight accuracy improvements and more stable outcomes were observed in deeper models of 7 layers and surprisingly the model with 3 layers and dropout 0.2 → 0.2→0.1 but it maybe it demands more computational data.

## SKIN CANCER CLASSIFICATION – MULTILAYER PERCEPTRONS (MLP) WITH PRINCIPAL COMPONENT ANALYSIS (PCA)

```
#Displaying 6 sample images from training dataset with their corresponding class labels
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
print("Training labels shape:", y_train.shape)

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(x_train[i])
    plt.title(f"Class: {np.argmax(y_train[i])}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

- This step prints the shape of the image data and value ranges to understand. Whether normalization is needed and what the input shape will be (e.g. 28x28 images).
- Visualizes 6 random training samples with their label (class) using imshow, as shown in the below picture:

```
y_train_labels = np.argmax(y_train, axis=1)
x_train_flat = x_train.reshape((x_train.shape[0], -1))
x_test_flat = x_test.reshape((x_test.shape[0], -1))
```

- unique: Displays number of samples per class to check class imbalance.
- Flatten the images to 1D vectors for PCA (e.g., 28x28x3 = 2352 features).
- Also flatten labels to match sklearn input requirements.

### 9. Class Imbalance Handling with Class Weights:

```
#Compute Class weights

from sklearn.utils.class_weight import compute_class_weight
import numpy as np

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_labels),
    y=y_train_labels
)

# Create dictionary for Keras
class_weights_dict = dict(enumerate(class_weights))

print("Class Weights:", class_weights_dict)
```

- Computes class weights to handle imbalanced datasets during training.
- These can be used in .fit() to penalize underrepresented classes.

### 10. Number of Classes

```
# Number of classes
num_classes = len(np.unique(y_train_labels))
```

- Gets the number of target classes for output layer size.

### 11. Apply PCA for Dimensionality Reduction

```
#Apply PCA
pca = PCA(n_components=10)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)
```

- Reduces input feature size from thousands to just 10 components in this example.
- Helps train faster and reduces overfitting risk, but may lose spatial information.

### 12. One-hot Encoding:

```
y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test
```

- Required format for Keras to compute categorical loss during training.

### 13. MLP Model Design (example of 4 layers):

```
# Build MLP model
mlp_model = Sequential([
    Dense(1024, activation='relu', input_shape=(10,)),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

- Input shape = 10 (PCA-reduced features).
- Hidden layers: Deep architecture (1024 → 512 → 256 → 128 neurons).

- Dropout layers: Reduce overfitting by randomly deactivating neurons during training.
- Output: Softmax for multi-class classification.

**14. Compilation & Training of the model:**

```
#Compile model
mlp_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

mlp_model.fit(x_train_pca, y_train_oh, epochs=10, batch_size=64, validation_split=0.1)
```

- Optimizer: Adam is used for adaptive learning rate.
- Loss: categorical_crossentropy because it's multi-class classification.
- Validation Split: 10% of training data used for validation during training.
- Epochs & Batch Size: Reasonable defaults (10 epochs, batch size 64).

**CASES**

Principal component analysis, or PCA, reduces the number of dimensions in large datasets to principal components that retain most of the original information. It does this by transforming potentially correlated variables into a smaller set of variables, called principal components.

The goal of this section is to design, evaluate and employ a Multilayer Perceptron (MLP) model to classify images from the DermaMNIST dataset and to analyze the effects of feature dimensionality reduction, via PCA, and the complexity of the model architecture (such as the number and size of layers) on its performance with the training and test data.

By applying PCA, we target:

- **Goal:** Reduce feature dimensionality to 10 using PCA (Principal Component Analysis).
- **Why:** PCA helps reduce noise and computation time while retaining key image features.
- **Effect:** Original image size is 28x28 = 784 pixels. This reduces the feature vector to only 10 principal components.

**Trial 1: MLP 4-LAYERS PCA 10**

The first MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=10)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(1024, activation='relu', input_shape=(10,)),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers dimensionality is 1024→512→256→128, and dropout 0.4→0.3→0.3→0.3. PCA = 10. Below the evaluation results:

```
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - accuracy: 0.6319 - loss: 1.1226 - val_accuracy: 0.6876 - val_loss: 0.9241
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.6811 - loss: 0.8907 - val_accuracy: 0.6819 - val_loss: 0.8885
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - accuracy: 0.6872 - loss: 0.8751 - val_accuracy: 0.6876 - val_loss: 0.8705
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - accuracy: 0.6888 - loss: 0.8715 - val_accuracy: 0.6919 - val_loss: 0.8582
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.6976 - loss: 0.8302 - val_accuracy: 0.6862 - val_loss: 0.8737
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.6991 - loss: 0.8371 - val_accuracy: 0.6876 - val_loss: 0.8686
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - accuracy: 0.7040 - loss: 0.8325 - val_accuracy: 0.7004 - val_loss: 0.8655
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.6916 - loss: 0.8261 - val_accuracy: 0.6904 - val_loss: 0.8499
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.6989 - loss: 0.8333 - val_accuracy: 0.7019 - val_loss: 0.8486
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - accuracy: 0.7009 - loss: 0.8190 - val_accuracy: 0.6819 - val_loss: 0.8551
MLP Test accuracy: 0.7057
```

| Training Summary | |
|---|---|
| Testing Accuracy | 70.57% |
| Validation Accuracy | 68.19% (last epoch) |
| Validation Loss | Decreased from 0.9241 to 0.8551 over 10 epochs |
| Final Parameters | ~700 thousands |

Table 15. MLP model - 4layers, PCA10.

The results in Table 1 indicate no overfitting. PCA (n=10) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 70.04% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

### Trial 2: MLP 4-LAYERS PCA 15

The second MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=15)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(1024, activation='relu', input_shape=(15,)),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers dimensionality is 1024→512→256→128, and dropout 0.4→0.3→0.3→0.3. PCA = 15. Below the evaluation results:

```
 Total params: 706,311 (2.69 MB)
 Trainable params: 706,311 (2.69 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - accuracy: 0.6371 - loss: 1.1150 - val_accuracy: 0.6819 - val_loss: 0.9184
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.6744 - loss: 0.9056 - val_accuracy: 0.6833 - val_loss: 0.8866
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - accuracy: 0.6967 - loss: 0.8619 - val_accuracy: 0.6933 - val_loss: 0.8915
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - accuracy: 0.6834 - loss: 0.8639 - val_accuracy: 0.6919 - val_loss: 0.8600
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.7024 - loss: 0.8214 - val_accuracy: 0.7047 - val_loss: 0.8456
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.7099 - loss: 0.7966 - val_accuracy: 0.6919 - val_loss: 0.8632
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.7194 - loss: 0.7820 - val_accuracy: 0.6947 - val_loss: 0.8703
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.7177 - loss: 0.7780 - val_accuracy: 0.7076 - val_loss: 0.8459
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - accuracy: 0.7222 - loss: 0.7702 - val_accuracy: 0.7104 - val_loss: 0.8370
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.7196 - loss: 0.7689 - val_accuracy: 0.7104 - val_loss: 0.8491
MLP Test accuracy: 0.7072
```

| Training Summary | |
|---|---|
| Testing Accuracy | 70.72% |
| Validation Accuracy | 71.04% (last epoch) |
| Validation Loss | Decreased from 0.9184 to 0.8491 over 10 epochs |
| Final Parameters | ~706 thousands |

Table 16. MLP model - 4layers, PCA15.

The results in Table 2 indicate no overfitting. PCA (n=15) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 71.04% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

**Trial 3: MLP 4-LAYERS PCA 20**

The third MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=20)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(1024, activation='relu', input_shape=(20,)),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers dimensionality is 1024→512→256→128, and dropout 0.4→0.3→0.3→0.3. PCA = 20. Below the evaluation results:

```
Total params: 711,431 (2.71 MB)
Trainable params: 711,431 (2.71 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 5s 31ms/step - accuracy: 0.6411 - loss: 1.0855 - val_accuracy: 0.6805 - val_loss: 0.8963
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - accuracy: 0.6810 - loss: 0.8711 - val_accuracy: 0.6904 - val_loss: 0.8647
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - accuracy: 0.6897 - loss: 0.8523 - val_accuracy: 0.6976 - val_loss: 0.8261
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - accuracy: 0.6970 - loss: 0.8215 - val_accuracy: 0.7090 - val_loss: 0.8232
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 21ms/step - accuracy: 0.7067 - loss: 0.7829 - val_accuracy: 0.7104 - val_loss: 0.8195
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - accuracy: 0.7200 - loss: 0.7677 - val_accuracy: 0.7133 - val_loss: 0.8035
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.7147 - loss: 0.7571 - val_accuracy: 0.7090 - val_loss: 0.7985
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.7242 - loss: 0.7406 - val_accuracy: 0.7190 - val_loss: 0.8199
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.7278 - loss: 0.7299 - val_accuracy: 0.7247 - val_loss: 0.8048
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.7306 - loss: 0.7131 - val_accuracy: 0.7161 - val_loss: 0.8034
MLP Test accuracy: 0.7157
```

| Training Summary | |
|---|---|
| Testing Accuracy | 71.57% |
| Validation Accuracy | 71.61% (last epoch) |
| Validation Loss | Decreased from 0.8963 to 0.8043 over 10 epochs |
| Final Parameters | ~711 thousands |

Table 17. MLP model - 4layers, PCA20.

The results in Table 3 indicate no overfitting. PCA (n=20) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 72.47% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

### Trial 4: MLP 4-LAYERS PCA 100

The fourth MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=100)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(1024, activation='relu', input_shape=(100,)),
    Dropout(0.4),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers is 1024→512→256→128, and dropout 0.4→0.3→0.3→0.3 respectively, PCA=100. Below the evaluation results:

```
 Total params: 793,351 (3.03 MB)
 Trainable params: 793,351 (3.03 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - accuracy: 0.6344 - loss: 1.1273 - val_accuracy: 0.6819 - val_loss: 0.8640
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.6860 - loss: 0.8693 - val_accuracy: 0.6947 - val_loss: 0.8444
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.7101 - loss: 0.7731 - val_accuracy: 0.7118 - val_loss: 0.8063
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.7190 - loss: 0.7535 - val_accuracy: 0.7204 - val_loss: 0.7655
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.7342 - loss: 0.6908 - val_accuracy: 0.7304 - val_loss: 0.7498
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.7454 - loss: 0.6616 - val_accuracy: 0.7332 - val_loss: 0.7502
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.7604 - loss: 0.6367 - val_accuracy: 0.7261 - val_loss: 0.7453
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - accuracy: 0.7632 - loss: 0.6135 - val_accuracy: 0.7504 - val_loss: 0.7351
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - accuracy: 0.7883 - loss: 0.5699 - val_accuracy: 0.7233 - val_loss: 0.7405
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.8104 - loss: 0.5082 - val_accuracy: 0.7375 - val_loss: 0.7443
MLP Test accuracy: 0.7287
```

| Training Summary | |
|---|---|
| Testing Accuracy | 72.87% |
| Validation Accuracy | 73.75% (last epoch) |
| Validation Loss | Decreased from 0.8640 to 0.7443 over 10 epochs |
| Final Parameters | ~793 thousands |

Table 18.MLP model - 4layers, PCA100.

The results in Table 4 indicate no overfitting. PCA (n=100) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 75.04% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

**Trial 5: MLP 6-LAYERS PCA 10**

The fifth MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=10)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(10,)),
    Dropout(0.4),
    Dense(1024, activation='relu'),
    Dropout(0.3),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers dimensionality is:

2048→1024→512→256→128→64, and dropout

0.4→0.3→0.3→0.3→0.3→0.3. PCA = 10. Below the evaluation results:

```
 Total params: 2,818,439 (10.75 MB)
 Trainable params: 2,818,439 (10.75 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 11s 90ms/step - accuracy: 0.6101 - loss: 1.1873 - val_accuracy: 0.6676 - val_loss: 0.9719
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 73ms/step - accuracy: 0.6721 - loss: 0.9384 - val_accuracy: 0.6733 - val_loss: 0.9222
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 74ms/step - accuracy: 0.6687 - loss: 0.9094 - val_accuracy: 0.6762 - val_loss: 0.8903
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 6s 62ms/step - accuracy: 0.6853 - loss: 0.8940 - val_accuracy: 0.6833 - val_loss: 0.9131
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 11s 65ms/step - accuracy: 0.6793 - loss: 0.9044 - val_accuracy: 0.6919 - val_loss: 0.8845
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 68ms/step - accuracy: 0.7012 - loss: 0.8717 - val_accuracy: 0.6876 - val_loss: 0.8776
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 6s 64ms/step - accuracy: 0.6861 - loss: 0.8695 - val_accuracy: 0.6919 - val_loss: 0.8866
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 11s 71ms/step - accuracy: 0.7021 - loss: 0.8508 - val_accuracy: 0.6961 - val_loss: 0.8590
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 6s 61ms/step - accuracy: 0.7090 - loss: 0.8326 - val_accuracy: 0.6976 - val_loss: 0.8839
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 70ms/step - accuracy: 0.6987 - loss: 0.8541 - val_accuracy: 0.6990 - val_loss: 0.8865
MLP Test accuracy: 0.6943
```

| Training Summary | |
|---|---|
| Testing Accuracy | 69.43% |
| Validation Accuracy | 69.90% (last epoch) |
| Validation Loss | Decreased from 0.9719 to 0.8865 over 10 epochs |
| Final Parameters | ~2.9 million |

Table 19. MLP model - 6layers, PCA10.

The results in Table 5 indicate no overfitting. PCA (n=10) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 69.90% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

**Trial 6: MLP 6-LAYERS PCA 15**

The sixth MLP model was built using the following structure:

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=15)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(15,)),
    Dropout(0.4),
    Dense(1024, activation='relu'),
    Dropout(0.3),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers dimensionality is:

2048→1024→512→256→128→64, and dropout

0.4→0.3→0.3→0.3→0.3→0.3. PCA = 15. Below the evaluation results:

```
 Total params: 2,828,679 (10.79 MB)
 Trainable params: 2,828,679 (10.79 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 12s 87ms/step - accuracy: 0.6072 - loss: 1.2325 - val_accuracy: 0.6377 - val_loss: 0.9699
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 9s 79ms/step - accuracy: 0.6660 - loss: 0.9262 - val_accuracy: 0.6748 - val_loss: 0.9052
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 73ms/step - accuracy: 0.6747 - loss: 0.9197 - val_accuracy: 0.6961 - val_loss: 0.8901
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 84ms/step - accuracy: 0.6859 - loss: 0.8791 - val_accuracy: 0.7004 - val_loss: 0.8734
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 71ms/step - accuracy: 0.6966 - loss: 0.8543 - val_accuracy: 0.7019 - val_loss: 0.8677
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 81ms/step - accuracy: 0.7001 - loss: 0.8398 - val_accuracy: 0.7090 - val_loss: 0.8497
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 72ms/step - accuracy: 0.7070 - loss: 0.8349 - val_accuracy: 0.7061 - val_loss: 0.8710
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 10s 72ms/step - accuracy: 0.7108 - loss: 0.8087 - val_accuracy: 0.7047 - val_loss: 0.8493
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 10s 73ms/step - accuracy: 0.7059 - loss: 0.8136 - val_accuracy: 0.7033 - val_loss: 0.8568
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 11s 84ms/step - accuracy: 0.7135 - loss: 0.7942 - val_accuracy: 0.7104 - val_loss: 0.8491
MLP Test accuracy: 0.7112
```

| Training Summary | |
|---|---|
| Testing Accuracy | 71.12% |
| Validation Accuracy | 71.04% (last epoch) |
| Validation Loss | Decreased from 0.9699 to 0.8491 over 10 epochs |
| Final Parameters | ~2.9 million |

Table 20. MLP model - 6layers, PCA15.

The results in Table 6 indicate no overfitting. PCA (n=15) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 71.04% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

### Trial 7: MLP 6-LAYERS PCA 20

The seventh MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=20)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(2048, activation='relu', input_shape=(20,)),
    Dropout(0.4),
    Dense(1024, activation='relu'),
    Dropout(0.3),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers dimensionality is:

2048→1024→512→256→128→64, and dropout

0.4→0.3→0.3→0.3→0.3→0.3. PCA = 20. Below the evaluation results:

```
Total params: 2,838,919 (10.83 MB)
Trainable params: 2,838,919 (10.83 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 12s 90ms/step - accuracy: 0.6216 - loss: 1.2129 - val_accuracy: 0.6633 - val_loss: 0.8949
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 73ms/step - accuracy: 0.6644 - loss: 0.9386 - val_accuracy: 0.6805 - val_loss: 0.9159
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 79ms/step - accuracy: 0.6946 - loss: 0.8657 - val_accuracy: 0.6976 - val_loss: 0.8513
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 69ms/step - accuracy: 0.6931 - loss: 0.8573 - val_accuracy: 0.7061 - val_loss: 0.8387
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 81ms/step - accuracy: 0.6855 - loss: 0.8615 - val_accuracy: 0.7133 - val_loss: 0.8486
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 10s 83ms/step - accuracy: 0.6985 - loss: 0.8225 - val_accuracy: 0.7161 - val_loss: 0.8313
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 72ms/step - accuracy: 0.7280 - loss: 0.7667 - val_accuracy: 0.7061 - val_loss: 0.8258
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 81ms/step - accuracy: 0.7006 - loss: 0.8092 - val_accuracy: 0.7104 - val_loss: 0.8426
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 7s 71ms/step - accuracy: 0.7166 - loss: 0.7637 - val_accuracy: 0.7218 - val_loss: 0.8328
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 8s 76ms/step - accuracy: 0.7197 - loss: 0.7579 - val_accuracy: 0.7076 - val_loss: 0.8200
MLP Test accuracy: 0.7142
```

| Training Summary | |
|---|---|
| Training Accuracy | 71.42% |
| Validation Accuracy | 70.76% (last epoch) |
| Validation Loss | Decreased from 0.8949 to 0.82 over 10 epochs |
| Final Parameters | ~2.9 million |

Table 21. MLP model - 6layers, PCA20.

The results in Table 7 indicate no overfitting. PCA (n=20) reduced computation and noise. The deep MLP successfully extracted patterns even with just 10 features. Despite the low-dimensional input, the model achieved a training accuracy of 72.18% (higher value), matching training accuracy closely, which suggests good generalization. This model has given the highest test accuracy yet and has a stable performance where val_loss decreases and training accuracy and val_accuracy are increasing (mainly).

### Trial 8: MLP 1-LAYER NO PCA

The eighth MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(32, activation='relu',input_shape=(x_train_flat.shape[1],) ),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layer is 32, and dropout 0.2, without PCA. This means that the entire image is used as input. Below the evaluation results:

```
 Total params: 75,527 (295.03 KB)
 Trainable params: 75,527 (295.03 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.6237 - loss: 1.2428 - val_accuracy: 0.6633 - val_loss: 1.0592
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6691 - loss: 1.0315 - val_accuracy: 0.6476 - val_loss: 1.0928
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6665 - loss: 1.0349 - val_accuracy: 0.6576 - val_loss: 1.0247
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6615 - loss: 1.0340 - val_accuracy: 0.6633 - val_loss: 0.9911
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6726 - loss: 1.0094 - val_accuracy: 0.6648 - val_loss: 0.9916
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6721 - loss: 0.9834 - val_accuracy: 0.6633 - val_loss: 0.9718
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6713 - loss: 0.9935 - val_accuracy: 0.6633 - val_loss: 0.9810
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6622 - loss: 1.0048 - val_accuracy: 0.6633 - val_loss: 0.9613
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6733 - loss: 0.9785 - val_accuracy: 0.6633 - val_loss: 0.9528
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6690 - loss: 0.9547 - val_accuracy: 0.6633 - val_loss: 0.9478
MLP Test accuracy: 0.6688
```

| Training Summary | |
|---|---|
| Testing Accuracy | 66.88% |
| Validation Accuracy | 66.33% (last epoch) |
| Validation Loss | Decreased from 1.0592 to 0.9478 over 10 epochs |
| Final Parameters | ~75 thousands |

Table 22. MLP model - 1layer, No PCA.

The results in Table 8 indicate no overfitting. Training accuracy increases slowly and steadily from 62.37% to 67.33% (highest accuracy). Validation accuracy remails almost steady at 66.33%. Loss decreases, which indicates that the model is learning, but performance is limited.

**Trial 9: MLP 1-LAYER PCA=100**

The ninth MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=100)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(32, activation='relu', input_shape=(100,)),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layer is 32, and dropout 0.2, PCA=100. Below the evaluation results:

```
 Total params: 3,463 (13.53 KB)
 Trainable params: 3,463 (13.53 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.4507 - loss: 1.5504 - val_accuracy: 0.6648 - val_loss: 1.0757
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6714 - loss: 1.0347 - val_accuracy: 0.6776 - val_loss: 0.9633
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6715 - loss: 0.9265 - val_accuracy: 0.6862 - val_loss: 0.9189
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6834 - loss: 0.8846 - val_accuracy: 0.6976 - val_loss: 0.8959
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6968 - loss: 0.8440 - val_accuracy: 0.7004 - val_loss: 0.8775
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6931 - loss: 0.8490 - val_accuracy: 0.7004 - val_loss: 0.8657
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7050 - loss: 0.8138 - val_accuracy: 0.6990 - val_loss: 0.8548
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7096 - loss: 0.8093 - val_accuracy: 0.6976 - val_loss: 0.8461
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7099 - loss: 0.7851 - val_accuracy: 0.7033 - val_loss: 0.8374
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7296 - loss: 0.7569 - val_accuracy: 0.7033 - val_loss: 0.8325
MLP Test accuracy: 0.7137
```

| Training Summary | |
|---|---|
| Testing Accuracy | 71.37% |
| Validation Accuracy | 70.33% (last epoch) |
| Validation Loss | Decreased from 1.0757 to 0.8325 over 10 epochs |
| Final Parameters | ~3.4 thousands |

Table 23. MLP model - 1layer, PCA100.

With a hidden layer of 32 units, the single-layer MLP model with 100 PCA components performed similarly on both training and validation accuracy. Validation loss was reduced and confirmed stable and efficient learning, as well as consistent training. It is likely that the input size is reduced, due to PCA, helping the model to train better. The validation accuracy improved from early training to a peak of 70.33% by the final epoch. Test accuracy of 71.37% confirms that the model generalizes well and is not overfitting.

**Trial 10: MLP 2-LAYERS PCA=100**

The tenth MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=100)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(64, activation='relu', input_shape=(100,)),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers is 64→32, and dropout 0.2 in both layers, PCA=100. Below the evaluation results:

```
 Total params: 8,775 (34.28 KB)
 Trainable params: 8,775 (34.28 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.5358 - loss: 1.3870 - val_accuracy: 0.6648 - val_loss: 0.9540
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6776 - loss: 0.9500 - val_accuracy: 0.6847 - val_loss: 0.8963
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6798 - loss: 0.9013 - val_accuracy: 0.6876 - val_loss: 0.8742
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6852 - loss: 0.8546 - val_accuracy: 0.7033 - val_loss: 0.8515
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6959 - loss: 0.8297 - val_accuracy: 0.7019 - val_loss: 0.8376
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7140 - loss: 0.7961 - val_accuracy: 0.7076 - val_loss: 0.8230
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.7061 - loss: 0.7999 - val_accuracy: 0.7133 - val_loss: 0.8184
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7071 - loss: 0.7826 - val_accuracy: 0.7090 - val_loss: 0.8105
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.7188 - loss: 0.7563 - val_accuracy: 0.7161 - val_loss: 0.8039
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.7165 - loss: 0.7603 - val_accuracy: 0.7218 - val_loss: 0.7968
MLP Test accuracy: 0.7237
```

| Training Summary | |
|---|---|
| Testing Accuracy | 72.37% |
| Validation Accuracy | 72.18% |
| Validation Loss | Decreased from 0.9540 to 0.7968 over 10 epochs |
| Final Parameters | ~8.8 thousands |

Table 24. MLP model - 2layers, PCA100.

This MLP architecture with approximately 8.8K parameters demonstrates strong classification performance on the given dataset. With a final test accuracy of 72.37%, the model achieves a well-balanced trade-off between complexity and generalization. Loss decreases, which indicates that the model is learning, but performance is limited.

### Trial 11: MLP 3-LAYERS PCA=100

The eleventh MLP model was built using the following structure:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten

# Number of classes
num_classes = len(np.unique(y_train_labels))

#Apply PCA
pca = PCA(n_components=100)
x_train_pca = pca.fit_transform(x_train_flat)
x_test_pca = pca.transform(x_test_flat)

from tensorflow.keras.utils import to_categorical

y_train_oh= to_categorical(y_train_labels, num_classes=num_classes)
y_test_oh = y_test

# Build MLP model
mlp_model = Sequential([
    Dense(512, activation='relu', input_shape=(100,)),
    Dropout(0.4),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

mlp_model.summary()
```

The architecture of MLP layers is 512→256→128, and dropout 0.4→0.3→0.3 respectively, PCA=100. Below the evaluation results:

```
 Total params: 216,839 (847.03 KB)
 Trainable params: 216,839 (847.03 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - accuracy: 0.6323 - loss: 1.1391 - val_accuracy: 0.6933 - val_loss: 0.8871
Epoch 2/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.6847 - loss: 0.8655 - val_accuracy: 0.6976 - val_loss: 0.8274
Epoch 3/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7029 - loss: 0.7950 - val_accuracy: 0.7133 - val_loss: 0.7944
Epoch 4/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7245 - loss: 0.7568 - val_accuracy: 0.7175 - val_loss: 0.7747
Epoch 5/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7141 - loss: 0.7474 - val_accuracy: 0.7332 - val_loss: 0.7630
Epoch 6/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7412 - loss: 0.6879 - val_accuracy: 0.7361 - val_loss: 0.7367
Epoch 7/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.7518 - loss: 0.6531 - val_accuracy: 0.7489 - val_loss: 0.7238
Epoch 8/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.7681 - loss: 0.6247 - val_accuracy: 0.7332 - val_loss: 0.7303
Epoch 9/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.7808 - loss: 0.6105 - val_accuracy: 0.7204 - val_loss: 0.7348
Epoch 10/10
99/99 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7816 - loss: 0.5868 - val_accuracy: 0.7475 - val_loss: 0.7206
MLP Test accuracy: 0.7337
```

Training Summary

| | |
|---|---|
| Testing Accuracy | 73.37% |
| Validation Accuracy | 74.75% |
| Validation Loss | Decreased from 0.8871 to 0.7206 over 10 epochs |
| Final Parameters | ~217 thousands |

Table 25.MLP model - 3layers, PCA100.

Validation accuracy improved steadily throughout training, with slight fluctuations near convergence. Final test accuracy (73.37%) confirms generalization, closely aligned with final validation accuracy. Validation loss decreased from 0.8871 → 0.7206, suggesting effective learning and no overfitting. Dropout helped mitigate overfitting despite moderate model complexity (~217k parameters).

## CONCLUSION

Even though multiple combinations of models were conducted, with variation of dimensionality (1 to 6 layers) and dropout (0.2 to 0.5) as well as PCA components (10, 15, 20, 100), we conclude the following:

- Best model → Trial 11: MLP 3-Layers & PCA=100 → highest testing accuracy 73.37%, validation accuracy 74.75%, validation loss decreased to 0.7206 (lowest outcome) & 217k parameters. This model offers the highest test accuracy, maintains stability during training, and generalizes well without overfitting. It is also significantly more efficient than deeper 6-layer models (~2.9M parameters) but still powerful enough to outperform shallower 1- or 2-layer models.

- Simplest model → Trial 10: MLP 2-Layers & PCA=100 → testing accuracy 72.37%, validation accuracy 72.18% & 8.8k parameters. Delivers high performance at a fraction of the parameter cost.

- Simplest model without PCA → Trial 8: MLP 1-Layer & No PCA → testing accuracy 72.37% & 75k parameters. When PCA isn't possible, this model still achieves reasonable accuracy with minimal structure.

- Training accuracy is performing close to validation accuracy, which has as an outcome the stability of the model and the avoidance of overfitting. It also confirms the effectiveness of dropout and PCA as regularization techniques. Validation loss is consistently decreasing in all models over 10 epochs, which reflects consistent and effective learning.

- PCA model performed equally well in all models, while not only reduced input size, but it likely improved the reduction of noise and redundancy, resulting in better generalization and faster training. PCA=100 gave the best balance of performance and input dimensionality. Very low PCA (10–20) still allowed deeper MLPs to extract patterns, but performance gains were modest compared to PCA=100.

# SKIN CANCER CLASSIFICATION-CONVOLUTIONAL NEURAL NETWORK (CNN)

Skin cancer is one of the most common forms of cancer worldwide, and its detection at an early stage is crucial. In this project, we consider a skin cancer image classification issue using deep learning models, i.e., Convolutional Neural Networks (CNNs), for skin lesion image classification from the DermaMNIST dataset. The dataset contains 7 classes of skin lesions, and the goal is to classify each image into its corresponding class correctly.

The dataset that was explored is a preprocessed MedMNIST dataset (test and train dataset). It is stored in .npz format with the below characteristics:

**1.Library Imports and Visualization:**

```
#Model Libraries:
import numpy as np
import pandas as pd
import os
from glob import glob
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPool2D,BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```
#Visualization Libraries:
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
import itertools
```

For library imports:

- numpy, pandas: Numerical and data handling.
- glob, os: File operations.
- tensorflow, keras: Deep learning framework.
- ImageDataGenerator: For image augmentation.
- train_test_split: To divide data into train/validation sets.

- to_categorical: Converts class labels into one-hot vectors.
- ReduceLROnPlateau: Reduces learning rate when a metric has stopped improving.

For data visualization:

- %matplotlib inline: Ensures plots show up in notebooks.
- matplotlib, seaborn: Data visualization.
- confusion_matrix: Evaluating classification performance.

2. **Data Loading and Preprocessing:**

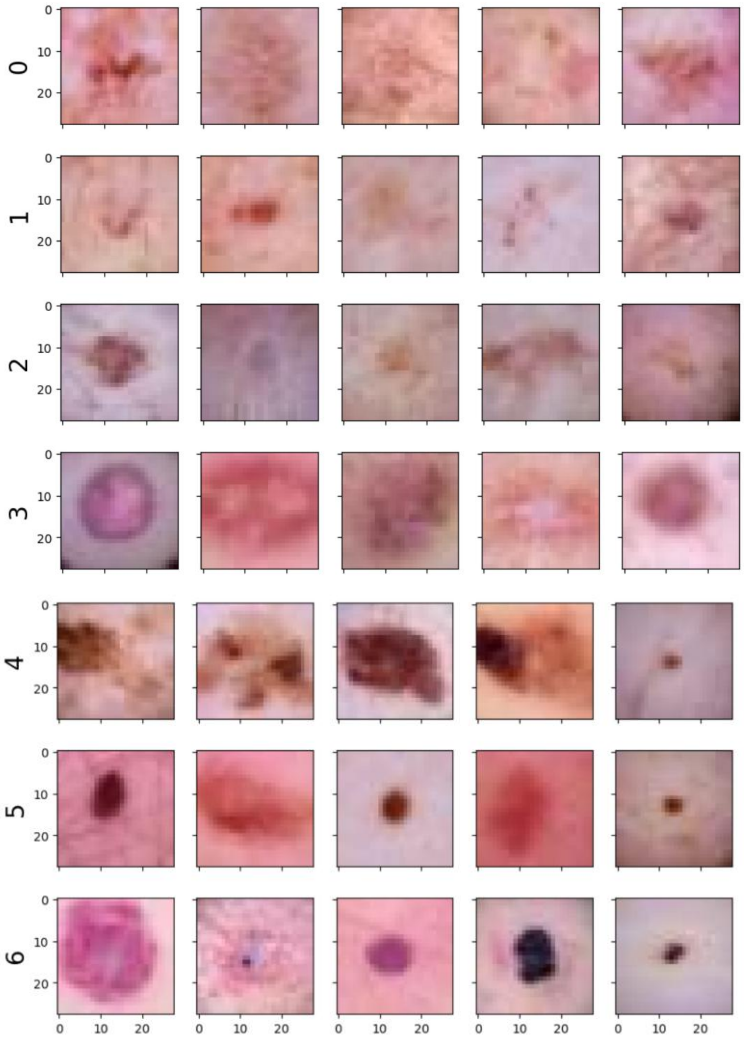
```
# Load and prepare data
train_data = np.load('DermaMNIST_Train.npz')
test_data = np.load('DermaMNIST_Test.npz')

x_train = train_data['x_train']
y_train = train_data['y_train']
x_test = test_data['x_test']
y_test = test_data['y_test']
#x_test = x_test / 255.0  # Normalize test data
x = np.concatenate((x_train, x_test), axis=0)
y= np.concatenate((y_train, y_test), axis=0)
```

- train_data, test_data: Loads the preprocessed .npz files containing training and test images and labels.
- x_train, x_test: Images of shape (28, 28, 3).
- y_train, y_test: One-hot encoded labels of shape (samples →7).
- x=, y=: Combine training and test data for initial visualization and inspection.

**3.Class Labels and Visualization:**

```
#Dictionary
labels = ['0',
          '1',
          '2',
          '3',
          '4',
          '5',
          '6']

lesion_type_dict = {
    '0': 'Actinic keratoses',
    '1': 'Basal cell carcinoma',
    '2': 'Benign keratosis-like lesions ',
    '3': 'Dermatofibroma',
    '4': 'Melanocytic nevi',
    '5': 'Vascular lesions',
    '6': 'Melanoma'
}

#Displaying 6 sample images from training dataset with their corresponding class labels
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
print("Training labels shape:", y_train.shape)

# Convert one-hot encoded labels to class indices
y_indices = np.argmax(y, axis=1)
y_train_indices=np.argmax(y_train, axis=1)

#Visualize the data
fig, ax = plt.subplots(7, 5)
fig.set_figheight(15)
fig.set_figwidth(10)
for classes in range (7):
  for i, inx in enumerate(np.where(y_indices==classes)[0][:5]):
    ax[classes,i].imshow(x[inx])
    ax[classes,i].set_ylabel(labels[classes],fontsize = 20.0)
    ax[classes,i].label_outer()
```

- Defining label names and their corresponding skin lesion categories.
- Converts one-hot labels into class indices (e.g., [0, 0, 1] → 1).
- Displays 5 sample images from each class for visual inspection.




## 4. Class Distribution:

```
num_classes = []
for i in range(len(labels)):
  num_classes.append(len(np.where(y_train_indices==i)[0]))

training_samples=pd.DataFrame(num_classes,index=labels)
print("\nSamples per class (training set):")
print(training_samples)
```

- Counts the number of samples in each class (for full and training sets).

## 5. Balancing the Dataset:

Skin lesions data are typically imbalanced, so both oversampling and undersampling are applied:

```python
from imblearn.over_sampling import RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler

os_dict = {0: 2000, #Increase minority classes
           1: 1500,
           2: 1500,
           3: 2500, #Increase minority classes
           4: 1500,
           5: num_classes[5],
           6: 2500} #Increase minority classes
oversample = RandomOverSampler(sampling_strategy=os_dict) # randomly duplicates existing samples from minority classes to meet the desired count
oversampled_x , oversampled_y = oversample.fit_resample(x_train.reshape(x_train.shape[0], -1), y_train_indices)
print('OS_x:%s OS_y:%s' %(oversampled_x.shape, oversampled_y.shape))

us_dict = {0: 2000,
           1: 1500,
           2: 1500,
           3: 2500,
           4: 1500,
           5: 3000, # reducing class 5
           6: 2500}

undersample = RandomUnderSampler(sampling_strategy=us_dict) #removes excess samples randomly from overrepresented classes
undersampled_x , undersampled_y = undersample.fit_resample(oversampled_x, oversampled_y)
print('US_x:%s US_y:%s' %(undersampled_x.shape, undersampled_y.shape))
```

```python
#Reshape back to image format
#For the previous command image had to be flatten from shape (28, 28, 3) to (28×28×3,) for each image (a 1D vector) where 2352 = 28×28×3.
undersampled_x = undersampled_x.reshape(-1,28,28,3)

#Count samples per class in the undersampled_y
num_classes = []
for i in range(len(labels)):
  num_classes.append(len(np.where(undersampled_y==i)[0]))

undersample_y=pd.DataFrame(num_classes,index=labels)
print("\nSamples per class (undersampled_y):")
print(undersample_y)

#Transforming y_label to one-hot encoded
y_train_under = to_categorical(undersampled_y) #undersampled_y --> undersampled y_train_indices
x_train_split, x_val_split, y_train_split, y_val_split = train_test_split(
    undersampled_x, y_train_under, test_size=0.2, random_state=42, stratify=undersampled_y)

print(y_train_under.shape)
print(y_test.shape) #already one-hot encoded
```

- Over-Sampling: Balances minority classes by duplicating samples. Also, images are reshaped to 1D before oversampling.
- Undersampling: Reduces samples of majority classes to avoid bias.
- undersampled_x: Reshape to images and restores the flattened images to original 3D format.
- One-Hot encoding and train/validation split (y_train_under , x_train_split, x_val_split, y_train_split, y_val_split): One-hot encodes the balanced labels. It also splits into 80% train and 20% validation sets.

## 6. Data Augmentation:

```python
train_datagen = ImageDataGenerator(rescale = 1./255, #Scales pixel values from [0, 255] to [0, 1] for better neural network stability
                                   rotation_range = 10,
                                   width_shift_range = 0.2,
                                   height_shift_range = 0.2,
                                   shear_range = 0.2, #Applies shearing transformations (tilting the image)
                                   horizontal_flip = True,
                                   vertical_flip = True,
                                   fill_mode = 'wrap')

batch_size=50
train_data = train_datagen.flow(undersampled_x, y_train_under, batch_size = batch_size, seed=1)
test_datagen = ImageDataGenerator(rescale=1./255)
test_data = test_datagen.flow(x_test, y_test, batch_size = batch_size, seed=1)
val_data = test_datagen.flow(x_val_split, y_val_split, batch_size=batch_size, seed=1)

print(f'Train Set: X:%s Y:%s' %(train_data.x.shape, train_data.y.shape))
print(f'Test Set : X:%s Y:%s' %(x_test.shape, y_test.shape))
```

- Generates new image variations to reduce overfitting and improve generalization.
- Applies augmentations to training data and normalizes all datasets.

## 7. CNN Model Architecture:

```
# Learning rate
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3)

#CNN model creation
num_classes = 7
cnn_model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 3)),
    BatchNormalization(),
    MaxPool2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    BatchNormalization(),
    MaxPool2D(pool_size=(2, 2)),

    Conv2D(128, (3, 3), activation='relu'),
    BatchNormalization(),

    Conv2D(256, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPool2D(pool_size=(2, 2)),

    Flatten(),

    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

cnn_model.summary()
```

- reduce_lr = It is a dynamic learning rate that improves training stability and performance. It monitors val_loss after every epoch. If val_loss doesn't improve after 3 epochs it will reduce the learning rate by 70%.
- A classic deep CNN with 4 convolutional blocks (32→64→128→256) and 3 fully connected layers (256→128→64).
- Flatten(): transforms a multi-dimensional tensor (like a 2D image or a 3D feature map) into a 1D vector.
- Uses batch normalization and dropout for regularization (256→128→64 & dropout 0.5→0.5→0.3 respectively).
- Final layer (64) uses softmax for multi-class classification.

## 8. Compilation and Training:

```
# Compile model
cnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model using the generator
history = cnn_model.fit(
    train_data,
    epochs=20,
    steps_per_epoch=len(train_data),
    validation_data=val_data,
    validation_steps=len(val_data),
    callbacks= reduce_lr,
)
```

- Optimizer: Adam (adaptive learning rate).

- Loss: Categorical crossentropy (for multi-class classification).
- history= : Trains the model for 20 epochs using the data generators.
- Includes a learning rate scheduler (ReduceLROnPlateau) that reduces the learning rate when validation loss stagnates.

## 9. Evaluation and Visualization:

```
# Evaluate the model
test_loss, test_acc = cnn_model.evaluate(test_data)
print(f"Test accuracy: {test_acc:.4f}")

# Plot CNN model
from tensorflow.keras.utils import plot_model
plot_model(cnn_model)
```

```
# Probability
y_proba = cnn_model.predict(x_test/255)
y_proba.round(2)

y_pred = np.argmax(y_proba, axis=-1)
y_pred[:10]

y_pred_name = np.array(labels)[y_pred]
y_pred_name[:10]

plt.figure(figsize=(20,10))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(x_test[i])
    plt.title('True label: {}, Prediction: {}'.format(labels[y_test[i].argmax()], y_pred_name[i]))
plt.tight_layout()

#Confusion matrix
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

y_test_numbers = np.array([y.argmax() for y in y_test])
cm = confusion_matrix(y_test_numbers, y_pred)
cm_display = ConfusionMatrixDisplay(cm,display_labels=labels)
fig, ax = plt.subplots(figsize=(10,10))
cm_display.plot(ax=ax)


#Classification report
print(classification_report(y_test_numbers, y_pred, target_names = labels))
```

- Evaluates model performance on test data.
- Visualizes model architecture as a flowchart.
- Designs a confusion matrix and a classification report

## RESULTS

The above code displays the results of the dataset. In the picture below we conclude the following:

```
Training data shape: (7007, 28, 28, 3)
Testing data shape: (2005, 28, 28, 3)
Training labels shape: (7007, 7)

Samples per class (whole dataset):
      0
0   294
1   462
2   989
3   103
4  1002
5  6034
6   128

Samples per class (training set):
      0
0   228
1   359
2   769
3    80
4   779
5  4693
6    99
OS_x:(16193, 2352) OS_y:(16193,)
US_x:(14500, 2352) US_y:(14500,)

Samples per class (undersampled_y):
      0
0  2000
1  1500
2  1500
3  2500
4  1500
5  3000
6  2500
(14500, 7)
(2005, 7)
Train Set: X:(14500, 28, 28, 3) Y:(14500, 7)
Test Set : X:(2005, 28, 28, 3) Y:(2005, 7)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, pre
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

a. Training images→7007 & Testing images→2005.
b. Each image is displayed in 28x28 pixels with 3 color channels→RGB.
c. The labels are one-hot encoded of 7 length, meaning the 7 classes.
d. Samples per class:

| *Classes* | Whole Dataset | Training Set |
|---|---|---|
| *0* | 294 | 228 |
| *1* | 462 | 359 |
| *2* | 989 | 769 |
| *3* | 103 | 80 |
| *4* | 1002 | 779 |
| *5* | 6034 | 4693 |
| *6* | 128 | 99 |

**Whole Dataset results:** Class 5 has 6034 samples, which is ~70% of your dataset, which indicates that is heavily imbalanced. Classes like 3 and 6 are extremely underrepresented (103 and 128 samples respectively).

**Training Set results:** These are the class counts after you split the full data into training and testing. The imbalance persists in the training set.

e. Over-Sampling and Under-Sampling:

| *OS_x* | 16193 | 2352 |
|---|---|---|
| *OS_y* | 16193, | |
| *US_x* | 14500 | 2352 |
| *US_y* | 14500 | |

- **Oversampling (OS):** Created 16,193 training samples by duplicating underrepresented classes. Feature size is 2352 (i.e., flattened from 28×28×3).

- **Undersampling (US):** Reduced the dataset to 14,500 samples by removing excess samples from overrepresented classes. This version is better for training a balanced model.

f. Class Distribution After Under-Sampling:

| *Classes* | Whole Dataset |
|---|---|
| *0* | 2000 |
| *1* | 1500 |
| *2* | 1500 |
| *3* | 1500 |
| *4* | 2000 |
| *5* | 3000 |
| *6* | 2000 |

Class 5 is still larger than the others, but now only 3000, which is reasonable. This ensures the model doesn't overly favor class 5.

g. Final Dataset Ready for Training and Testing:

**Train Set:** X:(14500, 28, 28, 3), y:(14500, 7)

**Test Set:** X:(2005, 28, 28, 3), y:(2005, 7)

After sampling, you reshaped and one-hot encoded the labels. The model of the dataset is now 14,500 images. Furthermore, testing is done on the 2005 examples.

## Model Training Summary & Learning Rate Behavior

We made multiple trials to find the best possible combination of maximum trial, validation and test accuracy and minimum validation accuracy as well. Some of those cases are interpreted below.

Case 1:

```
Total params: 497,735 (1.90 MB)
Trainable params: 496,775 (1.90 MB)
Non-trainable params: 960 (3.75 KB)
Epoch 1/20
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor.
  self._warn_if_super_not_called()
290/290 ━━━━━━━━━━━━━━━━━━━━ 49s 149ms/step - accuracy: 0.2555 - loss: 2.0664 - val_accuracy: 0.1724 - val_loss: 2.0605 - learning_rate: 0.0010
Epoch 2/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 144ms/step - accuracy: 0.3894 - loss: 1.5482 - val_accuracy: 0.2386 - val_loss: 2.6075 - learning_rate: 0.0010
Epoch 3/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 146ms/step - accuracy: 0.4574 - loss: 1.3641 - val_accuracy: 0.1841 - val_loss: 2.3277 - learning_rate: 0.0010
Epoch 4/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 81s 142ms/step - accuracy: 0.4988 - loss: 1.2775 - val_accuracy: 0.3259 - val_loss: 1.5970 - learning_rate: 0.0010
Epoch 5/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 143ms/step - accuracy: 0.5302 - loss: 1.2068 - val_accuracy: 0.3214 - val_loss: 1.7790 - learning_rate: 0.0010
Epoch 6/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 83s 145ms/step - accuracy: 0.5458 - loss: 1.1614 - val_accuracy: 0.2090 - val_loss: 2.6196 - learning_rate: 0.0010
Epoch 7/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 81s 142ms/step - accuracy: 0.5782 - loss: 1.1435 - val_accuracy: 0.1724 - val_loss: 2.7255 - learning_rate: 0.0010
Epoch 8/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 144ms/step - accuracy: 0.6143 - loss: 1.0595 - val_accuracy: 0.2572 - val_loss: 2.0540 - learning_rate: 3.0000e-04
Epoch 9/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 81s 142ms/step - accuracy: 0.6362 - loss: 1.0002 - val_accuracy: 0.4862 - val_loss: 1.3949 - learning_rate: 3.0000e-04
Epoch 10/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 142ms/step - accuracy: 0.6461 - loss: 0.9707 - val_accuracy: 0.3455 - val_loss: 15.4777 - learning_rate: 3.0000e-04
Epoch 11/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 142ms/step - accuracy: 0.6594 - loss: 0.9454 - val_accuracy: 0.3548 - val_loss: 1.7695 - learning_rate: 3.0000e-04
Epoch 12/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 83s 146ms/step - accuracy: 0.6697 - loss: 0.9156 - val_accuracy: 0.2993 - val_loss: 2.1814 - learning_rate: 3.0000e-04
Epoch 13/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 146ms/step - accuracy: 0.6768 - loss: 0.8964 - val_accuracy: 0.5655 - val_loss: 1.2207 - learning_rate: 9.0000e-05
Epoch 14/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 142ms/step - accuracy: 0.6960 - loss: 0.8410 - val_accuracy: 0.4741 - val_loss: 1.5961 - learning_rate: 9.0000e-05
Epoch 15/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 142ms/step - accuracy: 0.6988 - loss: 0.8225 - val_accuracy: 0.6848 - val_loss: 0.8403 - learning_rate: 9.0000e-05
Epoch 16/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 142ms/step - accuracy: 0.7071 - loss: 0.8229 - val_accuracy: 0.5276 - val_loss: 1.3565 - learning_rate: 9.0000e-05
Epoch 17/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 141ms/step - accuracy: 0.7026 - loss: 0.8210 - val_accuracy: 0.2941 - val_loss: 5.7745 - learning_rate: 9.0000e-05
Epoch 18/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 145ms/step - accuracy: 0.7038 - loss: 0.8100 - val_accuracy: 0.3283 - val_loss: 2.1846 - learning_rate: 9.0000e-05
Epoch 19/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 81s 141ms/step - accuracy: 0.7105 - loss: 0.7944 - val_accuracy: 0.6062 - val_loss: 1.1172 - learning_rate: 2.7000e-05
Epoch 20/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 146ms/step - accuracy: 0.7079 - loss: 0.7996 - val_accuracy: 0.6345 - val_loss: 1.2219 - learning_rate: 2.7000e-05
41/41 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - accuracy: 0.7005 - loss: 0.9018
Test accuracy: 0.7027
```

The CNN model has ~498k parameters (497,735) out of which almost 497k (496,775) are trainable. We employed manually a learning rate in order to optimize training. The learning rate started at 0.001, then it reduced to 0.0003, 0.0009 and lastly to 0.000027, which indicates more reliable and accurate weight updates in the following epochs. This schedule helped improve the model's validation performance without overfitting.

The model has a final training accuracy of ~71%, validation accuracy of ~64% and test accuracy of ~70%. Overall, this demonstrates a good generalization of the new data with reasonably moderate variance across training and test metrics. The learning rate schedule was crucial in attempting to balance the convergence speed and final performance.

Case 2:

```
Epoch 1/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 40s 121ms/step - accuracy: 0.2690 - loss: 2.0227 - val_accuracy: 0.1379 - val_loss: 2.5115 - learning_rate: 0.0010
Epoch 2/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 123ms/step - accuracy: 0.4309 - loss: 1.4621 - val_accuracy: 0.1724 - val_loss: 2.1094 - learning_rate: 0.0010
Epoch 3/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 125ms/step - accuracy: 0.4849 - loss: 1.3378 - val_accuracy: 0.1034 - val_loss: 9.5342 - learning_rate: 0.0010
Epoch 4/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 124ms/step - accuracy: 0.5155 - loss: 1.2656 - val_accuracy: 0.1724 - val_loss: 2.5986 - learning_rate: 0.0010
Epoch 5/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 124ms/step - accuracy: 0.5606 - loss: 1.1640 - val_accuracy: 0.1728 - val_loss: 9.9535 - learning_rate: 0.0010
Epoch 6/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 124ms/step - accuracy: 0.6005 - loss: 1.0752 - val_accuracy: 0.3707 - val_loss: 5.2680 - learning_rate: 3.0000e-04
Epoch 7/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 124ms/step - accuracy: 0.6279 - loss: 1.0141 - val_accuracy: 0.2648 - val_loss: 2.0754 - learning_rate: 3.0000e-04
Epoch 8/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 123ms/step - accuracy: 0.6448 - loss: 0.9842 - val_accuracy: 0.2197 - val_loss: 2.9514 - learning_rate: 3.0000e-04
Epoch 9/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 124ms/step - accuracy: 0.6452 - loss: 0.9649 - val_accuracy: 0.3245 - val_loss: 5.8014 - learning_rate: 3.0000e-04
Epoch 10/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 124ms/step - accuracy: 0.6579 - loss: 0.9266 - val_accuracy: 0.2503 - val_loss: 14.0851 - learning_rate: 3.0000e-04
Epoch 11/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 124ms/step - accuracy: 0.6693 - loss: 0.9009 - val_accuracy: 0.2466 - val_loss: 2.9256 - learning_rate: 9.0000e-05
Epoch 12/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 125ms/step - accuracy: 0.6844 - loss: 0.8767 - val_accuracy: 0.3510 - val_loss: 4.1505 - learning_rate: 9.0000e-05
Epoch 13/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 126ms/step - accuracy: 0.6949 - loss: 0.8436 - val_accuracy: 0.5179 - val_loss: 1.4167 - learning_rate: 9.0000e-05
Epoch 14/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 41s 125ms/step - accuracy: 0.6919 - loss: 0.8397 - val_accuracy: 0.5721 - val_loss: 1.1943 - learning_rate: 9.0000e-05
Epoch 15/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 124ms/step - accuracy: 0.6978 - loss: 0.8291 - val_accuracy: 0.5828 - val_loss: 1.9758 - learning_rate: 9.0000e-05
Epoch 16/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 35s 121ms/step - accuracy: 0.6985 - loss: 0.8242 - val_accuracy: 0.4731 - val_loss: 1.5306 - learning_rate: 9.0000e-05
Epoch 17/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 123ms/step - accuracy: 0.7073 - loss: 0.7980 - val_accuracy: 0.1966 - val_loss: 3.5387 - learning_rate: 9.0000e-05
Epoch 18/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 37s 129ms/step - accuracy: 0.7041 - loss: 0.7879 - val_accuracy: 0.5634 - val_loss: 1.3058 - learning_rate: 2.7000e-05
Epoch 19/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 125ms/step - accuracy: 0.7154 - loss: 0.7764 - val_accuracy: 0.5931 - val_loss: 1.0747 - learning_rate: 2.7000e-05
Epoch 20/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 36s 125ms/step - accuracy: 0.7205 - loss: 0.7727 - val_accuracy: 0.6414 - val_loss: 1.1377 - learning_rate: 2.7000e-05
41/41 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.7109 - loss: 0.8784
Test accuracy: 0.7152
```

The CNN model has ~498k parameters (497,735) out of which almost 497k (496,775) are trainable. We employed manually a learning rate to optimize training. The learning rate started at 0.001, then it reduced to 0.0003, 0.0009 and lastly to 0.000027, which indicates more reliable and accurate weight updates in the following epochs. This schedule helped improve the model's validation performance without overfitting.

The model has a final training accuracy of 72.05%, validation accuracy at 0.6414 and test accuracy of 71.52%. Overall, this demonstrates a good generalization of the new data with reasonably moderate variance across training and test metrics. The learning rate schedule was crucial in attempting to balance the convergence speed and final performance. The overall test accuracy is the higher accuracy we were able to reach with our model.

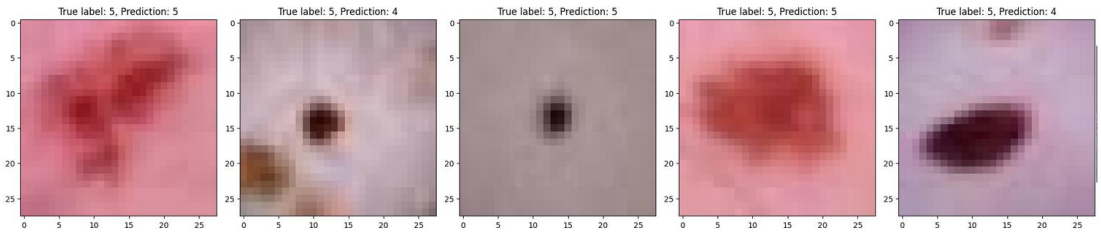
Case 3:

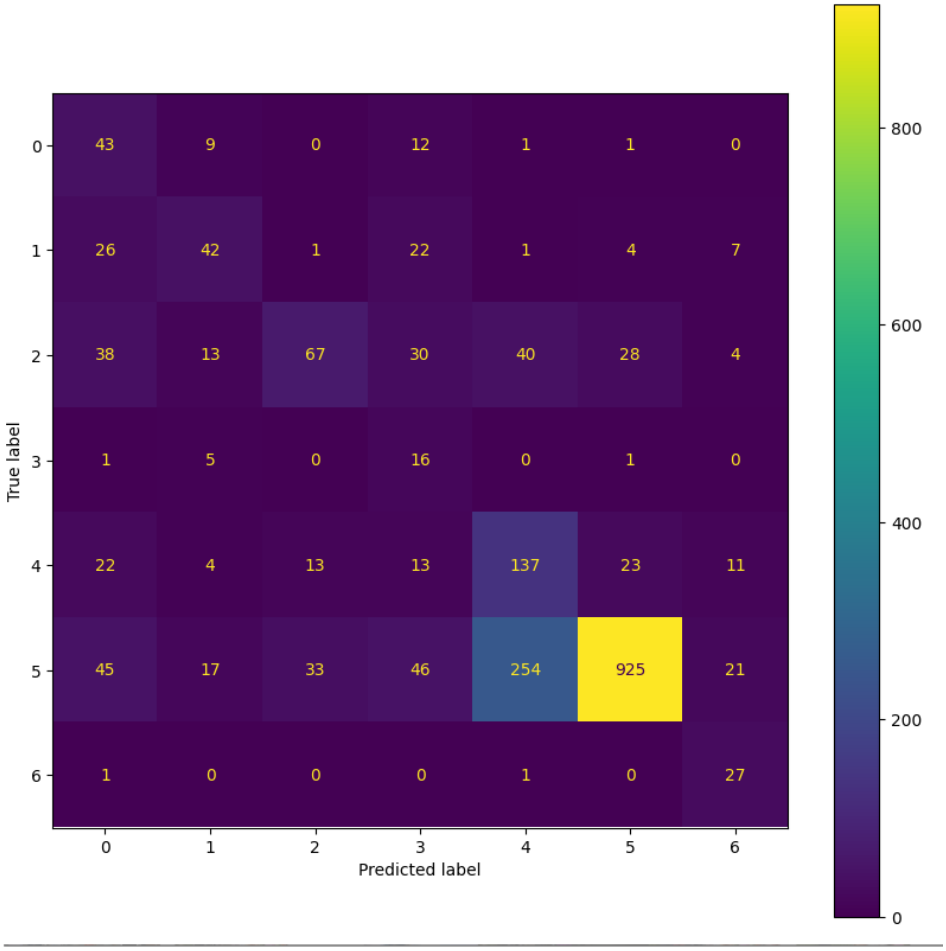
```
Epoch 1/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 48s 146ms/step - accuracy: 0.2347 - loss: 2.0564 - val_accuracy: 0.1379 - val_loss: 2.1361 - learning_rate: 0.0010
Epoch 2/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 44s 151ms/step - accuracy: 0.3904 - loss: 1.5496 - val_accuracy: 0.1690 - val_loss: 2.3254 - learning_rate: 0.0010
Epoch 3/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 150ms/step - accuracy: 0.4715 - loss: 1.3342 - val_accuracy: 0.2803 - val_loss: 3.5833 - learning_rate: 0.0010
Epoch 4/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 151ms/step - accuracy: 0.4929 - loss: 1.2781 - val_accuracy: 0.2066 - val_loss: 7.0081 - learning_rate: 0.0010
Epoch 5/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 143ms/step - accuracy: 0.5396 - loss: 1.1714 - val_accuracy: 0.2359 - val_loss: 1.7857 - learning_rate: 3.0000e-04
Epoch 6/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 144ms/step - accuracy: 0.5594 - loss: 1.1233 - val_accuracy: 0.1228 - val_loss: 4.2901 - learning_rate: 3.0000e-04
Epoch 7/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 83s 148ms/step - accuracy: 0.5861 - loss: 1.0951 - val_accuracy: 0.2569 - val_loss: 12.8443 - learning_rate: 3.0000e-04
Epoch 8/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 81s 145ms/step - accuracy: 0.5974 - loss: 1.0503 - val_accuracy: 0.2731 - val_loss: 1.8056 - learning_rate: 3.0000e-04
Epoch 9/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 87s 161ms/step - accuracy: 0.6198 - loss: 1.0124 - val_accuracy: 0.3562 - val_loss: 11.6147 - learning_rate: 9.0000e-05
Epoch 10/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 146ms/step - accuracy: 0.6340 - loss: 0.9955 - val_accuracy: 0.3707 - val_loss: 4.9358 - learning_rate: 9.0000e-05
Epoch 11/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 145ms/step - accuracy: 0.6282 - loss: 0.9960 - val_accuracy: 0.4772 - val_loss: 1.3868 - learning_rate: 9.0000e-05
Epoch 12/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 43s 147ms/step - accuracy: 0.6375 - loss: 0.9784 - val_accuracy: 0.1552 - val_loss: 3.9301 - learning_rate: 9.0000e-05
Epoch 13/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 148ms/step - accuracy: 0.6493 - loss: 0.9429 - val_accuracy: 0.5490 - val_loss: 1.4051 - learning_rate: 9.0000e-05
Epoch 14/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 144ms/step - accuracy: 0.6594 - loss: 0.9283 - val_accuracy: 0.3676 - val_loss: 2.8914 - learning_rate: 9.0000e-05
Epoch 15/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 43s 147ms/step - accuracy: 0.6509 - loss: 0.9422 - val_accuracy: 0.5848 - val_loss: 1.0872 - learning_rate: 2.7000e-05
Epoch 16/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 83s 149ms/step - accuracy: 0.6643 - loss: 0.8958 - val_accuracy: 0.6441 - val_loss: 0.9254 - learning_rate: 2.7000e-05
Epoch 17/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 148ms/step - accuracy: 0.6716 - loss: 0.8840 - val_accuracy: 0.6066 - val_loss: 1.0796 - learning_rate: 2.7000e-05
Epoch 18/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 81s 146ms/step - accuracy: 0.6682 - loss: 0.9024 - val_accuracy: 0.6679 - val_loss: 0.8707 - learning_rate: 2.7000e-05
Epoch 19/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 82s 148ms/step - accuracy: 0.6714 - loss: 0.9106 - val_accuracy: 0.4710 - val_loss: 1.4672 - learning_rate: 2.7000e-05
Epoch 20/20
290/290 ━━━━━━━━━━━━━━━━━━━━ 42s 143ms/step - accuracy: 0.6680 - loss: 0.9101 - val_accuracy: 0.6855 - val_loss: 0.8663 - learning_rate: 2.7000e-05
41/41 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - accuracy: 0.6833 - loss: 0.8033
Test accuracy: 0.6858
```

The CNN model has ~498k parameters (497,735) out of which almost 497k (496,775) are trainable. We employed manually a learning rate in order to optimize training. The learning rate started at 0.001, then it reduced to 0.0003, 0.0009 and lastly to 0.000027, which indicates more reliable and accurate weight updates in the following epochs. This schedule helped improve the model's validation performance without overfitting.

The model has a final training accuracy of ~68%, validation accuracy of ~68% and test accuracy of ~69%. Overall, this demonstrates a good generalization of the new data with reasonably moderate variance across training and test metrics. The learning rate schedule was crucial in attempting to balance the convergence speed and final performance.

Below is an example of the cross checking that the model does and prints for users of one of the trials we conducted. As the images show, the model was able to predict correctly 3 times out of 5 regarding the class 5 (which is the dominant class of the dataset).




Regarding the confusion matrix below, from one of the trials that were conducted, we can clearly see that despite the fact that for this specific trial the results are generally low, the majority of the predictions of each class are at the correct label

and there is no confusion that misleads the model. Definitely, there is room for improvement regarding the precision.